rqbit 데이터 검증 및 해시 확인 분석

개요

rqbit의 데이터 검증 시스템은 BitTorrent 프로토콜의 핵심 보안 메커니즘으로, SHA-1 해시를 통해 다운로드된 데이터의 무결성을 보장합니다. 이 시스템은 조각(piece) 단위로 데이터를 검증하며, 손상된 데이터를 감지하고 재다운로드를 트리거하는 기능을 제공합니다.

SHA-1 래퍼 시스템 (sha1w)

추상화 트레이트

```rust
pub trait ISha1 {

fn new() -> Self;

fn update(&mut self, buf: &[u8]);

fn finish(self) -> [u8; 20];

}
```

설계 목적:

- 다양한 SHA-1 구현체 간의 통일된 인터페이스 제공
- 성능 최적화를 위한 구현체 교체 용이성
- 플랫폼별 최적화된 라이브러리 활용

구현체 선택

1. crypto-hash 기반 구현

```rust
#[cfg(feature = "sha1-crypto-hash")]
mod crypto_hash_impl {
use super::ISha1;

pub struct Sha1CryptoHash {
inner: crypto_hash::Hasher,
}

impl ISha1 for Sha1CryptoHash {
fn new() -> Self {
Self {
inner: crypto_hash::Hasher::new(crypto_hash::Algorithm::SHA1),
}
}

fn update(&mut self, buf: &[u8]) {
use std::io::Write;
self.inner.write_all(buf).unwrap();
}

fn finish(mut self) -> [u8; 20] {
```

```
let result = self.inner.finish();

debug_assert_eq!(result.len(), 20);

let mut result_arr = [0u8; 20];

result_arr.copy_from_slice(&result);

result_arr

}

}

}
```

특징:

- 시스템 라이브러리 활용 (OpenSSL 등)
- 높은 성능 (OpenSSL은 순수 Rust 구현보다 2-3배 빠름)
- 기본 구현체로 사용

2. AWS LC (BoringSSL) 기반 구현

```
#[cfg(feature = "sha1-ring")]

mod ring_impl {

use super::ISha1;

use aws_lc_rs::digest::{Context, SHA1_FOR_LEGACY_USE_ONLY as SHA1};



pub struct Sha1Ring {

ctx: Context,

}
```

```
impl ISha1 for Sha1Ring {
fn new() -> Self {
Self {
ctx: Context::new(&SHA1),
}
}

fn update(&mut self, buf: &[u8]) {
self.ctx.update(buf);
}

fn finish(self) -> [u8; 20] {
let result = self.ctx.finish();
debug_assert_eq!(result.as_ref().len(), 20);
let mut result_arr = [0u8; 20];
result_arr.copy_from_slice(result.as_ref());
result_arr
}
}
}
```

특징:

- AWS의 BoringSSL 포크 사용

- 높은 보안성과 성능
- 대안 구현체로 제공

피처 플래그 시스템

```
[features]
default = ["sha1-crypto-hash"]
sha1-crypto-hash = ["crypto-hash"]
sha1-ring = ["aws-lc-rs"]
```

컴파일 타임 선택:

- `assert_cfg::exactly_one!` 매크로로 하나의 구현체만 선택 보장
- 조건부 컴파일을 통한 최적화된 바이너리 생성

토렌트 메타정보의 해시 관리

해시 저장 구조

```rust
pub struct TorrentMetaV1Info<BufType> {
pub pieces: BufType, // 연속된 20바이트 해시들
// ... 기타 필드들
}
```

저장 방식:

- 각 조각의 SHA-1 해시를 20바이트씩 연속으로 저장

- 조각 인덱스 × 20으로 해당 해시 위치 계산

해시 비교 메서드

```
impl<BufType: AsRef<[u8]>> TorrentMetaV1Info<BufType> {

pub fn get_hash(&self, piece: u32) -> Option<&[u8]> {

let start = piece as usize * 20;

let end = start + 20;

let expected_hash = self.pieces.as_ref().get(start..end)?;

Some(expected_hash)

}


pub fn compare_hash(&self, piece: u32, hash: [u8; 20]) -> Option<bool> {

let start = piece as usize * 20;

let end = start + 20;

let expected_hash = self.pieces.as_ref().get(start..end)?;

Some(expected_hash == hash)

}

}
```

안전성 보장:

- 범위 검사를 통한 버퍼 오버플로우 방지
- `Option` 반환으로 유효하지 않은 조각 인덱스 처리

파일 작업을 통한 해시 계산

update_hash_from_file 함수

```
pub fn update_hash_from_file<Sha1: ISha1>(
file_id: usize,
file_info: &FileInfo,
mut pos: u64,
files: &dyn TorrentStorage,
hash: &mut Sha1,
buf: &mut [u8],
mut bytes_to_read: usize,
) -> anyhow::Result<()> {
let mut read = 0;
while bytes_to_read > 0 {
let chunk = std::cmp::min(buf.len(), bytes_to_read);
if file_info.attrs.padding {
buf[..chunk].fill(0); // 패딩 파일은 0으로 채움
} else {
files
.pread_exact(file_id, pos, &mut buf[..chunk])
.with_context(|| {
format!("failed reading chunk of size {chunk}, read so far {read}")
```

```
})?;

}

bytes_to_read -= chunk;

read += chunk;

pos += chunk as u64;

hash.update(&buf[..chunk]); // 해시 업데이트

}

Ok(())

}
```

특징:

- 청크 단위로 파일 읽기 (메모리 효율성)
- 패딩 파일 특별 처리 (0으로 채움)
- 점진적 해시 계산
- 상세한 오류 컨텍스트 제공

조각 검증 시스템

개별 조각 검증 (check_piece)

```
pub fn check_piece(&self, piece_index: ValidPieceIndex) -> anyhow::Result<bool>

if cfg!(feature = "_disable_disk_write_net_benchmark") {

return Ok(true); // 벤치마크 모드에서는 검증 생략

}
```

```
let mut h = Sha1::new();
let piece_length = self.torrent.lengths().piece_length(piece_index);
let mut absolute_offset = self.torrent.lengths().piece_offset(piece_index);
let mut buf = vec![0u8; std::cmp::min(65536, piece_length as usize)];


let mut piece_remaining_bytes = piece_length as usize;


// 여러 파일에 걸친 조각 처리
for (file_idx, fi) in self.file_infos.iter().enumerate() {
let file_len = fi.len;
if absolute_offset > file_len {
absolute_offset -= file_len;
continue;
}
let file_remaining_len = file_len - absolute_offset;


let to_read_in_file: usize =
std::cmp::min(file_remaining_len, piece_remaining_bytes as u64).try_into()?;
update_hash_from_file(
file_idx,
fi,
absolute_offset,
```

```
self.files,
&mut h,
&mut buf,
to_read_in_file,
)?;

piece_remaining_bytes -= to_read_in_file;

if piece_remaining_bytes == 0 {
break;
}

absolute_offset = 0;
}

// 해시 비교 및 결과 처리
match self.torrent.info().compare_hash(piece_index.get(), h.finish()) {
Some(true) => {
trace!("piece={} hash matches", piece_index);
Ok(true)
}
Some(false) => {
let piece_length = self.torrent.lengths().piece_length(piece_index);
```

```
let absolute_offset = self.torrent.lengths().piece_offset(piece_index);

warn!(

piece_length,

absolute_offset, "the piece={} hash does not match", piece_index

);

Ok(false)

}

None => {

warn!("compare_hash() did not find the piece");

anyhow::bail!("compare_hash() did not find the piece");

}

}

}
```

검증 과정:

1. 조각의 절대 오프셋과 길이 계산
2. 여러 파일에 걸친 데이터 순차 읽기
3. 점진적 해시 계산
4. 예상 해시와 비교
5. 결과에 따른 로깅 및 반환

초기 검사 (initial_check)

```
pub fn initial_check(&self, progress: &AtomicU64) -> anyhow::Result<BF> {
```

```
let mut have_pieces =

BF::from_boxed_slice(vec![0u8; self.torrent.lengths().piece_bitfield_bytes()].in

let mut piece_files = Vec::<usize>::new();



#[derive(Debug)]

struct CurrentFile<'a> {

index: usize,

fi: &'a FileInfo,

processed_bytes: u64,

is_broken: bool,

}



let mut file_iterator = self

.file_infos

.iter()

.enumerate()

.map(|(idx, fi)| CurrentFile {

index: idx,

fi,

processed_bytes: 0,

is_broken: false,

});
```

```
let mut current_file = file_iterator.next().context("empty input file list")?;
let mut read_buffer = vec![0u8; 65536];


for piece_info in self.torrent.lengths().iter_piece_infos() {
piece_files.clear();
let mut computed_hash = Sha1::new();
let mut piece_remaining = piece_info.len as usize;
let mut some_files_broken = false;
progress.fetch_add(piece_info.len as u64, Ordering::Relaxed);


while piece_remaining > 0 {
let mut to_read_in_file: usize =
std::cmp::min(current_file.remaining(), piece_remaining as u64).try_into()?;


// 길이가 0인 파일 건너뛰기
while to_read_in_file == 0 {
current_file = file_iterator.next().context("broken torrent metadata")?;
to_read_in_file =
std::cmp::min(current_file.remaining(), piece_remaining as u64)
.try_into()?;
}


piece_files.push(current_file.index);
```

```
let pos = current_file.processed_bytes;

piece_remaining -= to_read_in_file;

current_file.mark_processed_bytes(to_read_in_file as u64);



if current_file.is_broken {

continue; // 손상된 파일은 건너뛰기

}



if let Err(err) = update_hash_from_file(

current_file.index,

current_file.fi,

pos,

self.files,

&mut computed_hash,

&mut read_buffer,

to_read_in_file,

) {

debug!(

"error reading from file {} ({:?}) at {}: {:#}",

current_file.index, current_file.fi.relative_filename, pos, &err

);

current_file.is_broken = true;
```

```
some_files_broken = true;
}
}

if some_files_broken {
trace!(
"piece {} had errors, marking as needed",
piece_info.piece_index
);
continue; // 손상된 조각은 다운로드 필요로 마킹
}

if self
.torrent
.info()
.compare_hash(piece_info.piece_index.get(), computed_hash.finish())
.context("bug: either torrent info broken or we have a bug - piece index invalid
{
have_pieces.set(piece_info.piece_index.get() as usize, true);
}
}

Ok(have_pieces)
```

```
}
```

초기 검사 특징:

- 모든 조각을 순차적으로 검증
- 파일 I/O 오류 허용 (손상된 파일 처리)
- 진행률 추적 (`AtomicU64` 사용)
- 비트벡터로 결과 반환

손상된 데이터 처리

손상된 조각 마킹

```
pub fn mark_piece_broken_if_not_have(&mut self, index: ValidPieceIndex) {
if self
.have
.as_slice()
.get(index.get() as usize)
.map(|r| *r)
.unwrap_or_default()
{
return; // 이미 보유한 조각은 건드리지 않음
}
debug!("marking piece={} as broken", index);
self.queue_pieces.set(index.get() as usize, true); // 대기열에 다시 추가
```

```
if let Some(s) = self.chunk_status.get_mut(self.lengths.chunk_range(index)) {
s.fill(false); // 청크 상태 초기화
}
}
```

복구 메커니즘:

1. 이미 검증된 조각은 보호
2. 대기열에 재추가하여 재다운로드 트리거
3. 청크 상태 초기화로 부분 다운로드 상태 제거

실시간 검증 실패 처리

```
// 라이브 토렌트 상태에서의 처리
match state.file_ops().check_piece(chunk_info.piece_index)? {
true => {
// 검증 성공 - 조각 완료 처리
state.lock_write("mark_piece_downloaded")
.get_chunks_mut()?
.mark_piece_downloaded(chunk_info.piece_index);
}
false => {
// 검증 실패 - 손상된 조각으로 마킹
warn!("piece {} failed hash check, marking as broken", chunk_info.piece_index);
state.lock_write("mark_piece_broken")
```

```
.get_chunks_mut()?
.mark_piece_broken_if_not_have(chunk_info.piece_index);
state.new_pieces_notify.notify_waiters();
anyhow::bail!("hash verification failed");
}
}
```

성능 최적화

1. 버퍼 크기 최적화

```
let mut buf = vec![0u8; std::cmp::min(65536, piece_length as usize)];
```

- 64KB 버퍼 사용 (I/O 효율성과 메모리 사용량 균형)
- 작은 조각의 경우 조각 크기에 맞춤

2. 점진적 해시 계산

- 전체 조각을 메모리에 로드하지 않고 스트리밍 방식으로 해시 계산
- 대용량 조각에 대한 메모리 효율성

3. 벤치마크 모드

```
if cfg!(feature = "_disable_disk_write_net_benchmark") {
return Ok(true);
```

```
}
```

- 성능 측정 시 해시 검증 비활성화 옵션
- 순수 네트워크/디스크 I/O 성능 측정 가능

4. 캐싱 및 최적화

- SHA-1 구현체 선택을 통한 성능 최적화
- 시스템 라이브러리 활용으로 하드웨어 가속 지원

오류 처리 및 복구

계층적 오류 처리

1. **파일 I/O 오류**: 파일을 손상된 것으로 마킹, 계속 진행
2. **해시 불일치**: 조각을 재다운로드 대상으로 마킹
3. **시스템 오류**: 상위 레벨로 전파

복구 전략

```
// 초기 검사에서의 오류 허용
if let Err(err) = update_hash_from_file(...) {
debug!("error reading from file: {:#}", err);
current_file.is_broken = true;
some_files_broken = true;
}
```

```
// 손상된 조각은 다운로드 필요로 마킹

if some_files_broken {

trace!("piece {} had errors, marking as needed", piece_info.piece_index);

continue;

}
```

보안 고려사항

1. 해시 검증 우회 방지

- 모든 수신 데이터에 대한 강제 검증
- 벤치마크 모드는 컴파일 타임에만 활성화

2. 타이밍 공격 방지

- 상수 시간 해시 비교 (구현체 의존)
- 해시 불일치 시에도 일관된 처리

3. 메모리 안전성

- 범위 검사를 통한 버퍼 오버플로우 방지
- Rust의 메모리 안전성 보장 활용

통계 및 모니터링

검증 통계

- 검증된 조각 수 추적

- 검증 실패 조각 수 모니터링
- 평균 검증 시간 측정

로깅

```
trace!("piece={} hash matches", piece_index);

warn!("the piece={} hash does not match", piece_index);

debug!("marking piece={} as broken", index);
```

- 상세한 검증 결과 로깅
- 성능 영향 최소화를 위한 레벨별 로깅

결론

rqbit의 데이터 검증 시스템은 다음과 같은 특징을 제공합니다:

1. **강력한 무결성 보장**: SHA-1 해시를 통한 조각별 검증
2. **성능 최적화**: 다양한 SHA-1 구현체 지원과 효율적인 I/O
3. **오류 복구**: 손상된 데이터 감지 및 자동 재다운로드
4. **메모리 효율성**: 스트리밍 방식의 해시 계산
5. **유연성**: 컴파일 타임 구현체 선택과 런타임 최적화

이러한 설계를 통해 rqbit은 BitTorrent 프로토콜의 보안 요구사항을 충족하면서도 높은 성능을 달성합니다.